
◆ What is CoAP (in simple words)?

- **CoAP (Constrained Application Protocol)** is a **lightweight web protocol**
- Used in:
 - IoT devices
 - Wireless Sensor Networks (WSN)
- Works on **UDP (faster but unreliable)**
- Similar to HTTP but **much lighter**

Think:

HTTP = heavy (for computers)

CoAP = lightweight (for small IoT devices)

◆ Key Idea

- Devices communicate using **Client–Server model**
 - Uses **REST methods**:
 - GET → read data
 - POST → create
 - PUT → update
 - DELETE → remove
-

◆ CoAP Communication (Step-by-Step)

✅ Step 1: Client Discovery

- Client wants to find servers
- Client sends a **multicast message (UDP Discovery)**
- Purpose: find available CoAP servers available on the network.

👉 Example:
“Any CoAP server here?”

✅ Step 2: Client Request

- Client sends request to server
 - **URI** (Uniform Resource Identifier): Identifies the target resource on the server.

Includes:

- **Method** → GET, POST, PUT, DELETE
- **URI** → resource (e.g., /temp)
- **Message ID** → unique number
- **Payload** → optional data Optional data to be sent along with the request.

👉 Example:

GET coap://server/temp

“Give me temperature from /temp”

✅ Step 3: Server Response

- Server processes request
- Sends back response

Includes:

- **Message ID** (same as request)
- **Status Code** (e.g., success, error)
- **Options** Additional parameters like content type, observe, etc.
- **Payload** (data)

👉 Example:

Temperature = 25°C

✅ Step 4: Confirmable (CON) Message

- Message that **requires acknowledgment**
- Used when reliability is important

👉 Client says:

“I need confirmation you received this”

CoAP messages can be either confirmable (CON) or non-confirmable (NON).

A CON message expects an acknowledgment (ACK) from the **recipient**. If the **client sends** a CON message, it **waits** for an ACK before proceeding.

✅ Step 5: Acknowledgment (ACK)

- Server sends ACK for CON message
- Confirms message delivery

👉 Server says:

“✓ I received your request”

When a CoAP server receives a CON message, it sends an ACK message back to the client to confirm message delivery. The ACK message contains the message ID of the received CON message.

✅ Step 6: Non-Confirmable (NON) Message

- No acknowledgment required
- Faster but less reliable

👉 Example:

“Just sending, no confirmation needed”

✓ Step 7: Separate Response

- If server needs time:
 1. First sends ACK
 2. Later sends actual response

👉 Example:

- Client: "Give data"
- Server: "OK received (ACK)"
- Later: "Here is data"

In some cases, the server may decide to send the response in a separate CoAP message. This can be useful when the server needs more time to process the request or when the response is too large to fit in a single message. The client expects the separate response and waits for it.

✓ Step 8: Retransmission

- If no ACK received:
 - Client assumes message lost
 - Resends message



👉 This adds **reliability over UDP**

If the client does not receive an acknowledgment (ACK) for a CON message, it assumes the message was lost.

The client retransmits the message. This ensures reliable delivery in the presence of unreliable networks

◆ Message Types Summary

- CON → needs ACK
- NON → no ACK

- ACK → confirmation
 - RESET → error / rejection
-

◆ Why CoAP is useful?

- Very **lightweight**
 - Saves **battery & bandwidth**
 - Supports:
 - REST (like HTTP)
 - Real-time updates (Observe feature)
 - Resource discovery
-

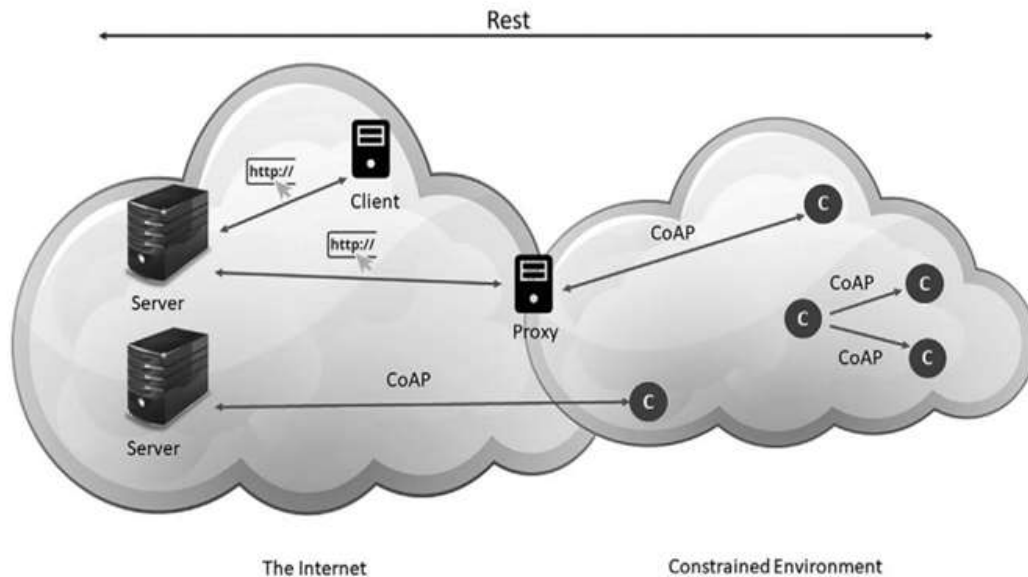
◆ Easy Real-Life Example 🧪

- Sensor (client) → sends request
 - Server → gives temperature
 - If important → uses CON + ACK
 - If not important → uses NON
-

Features:

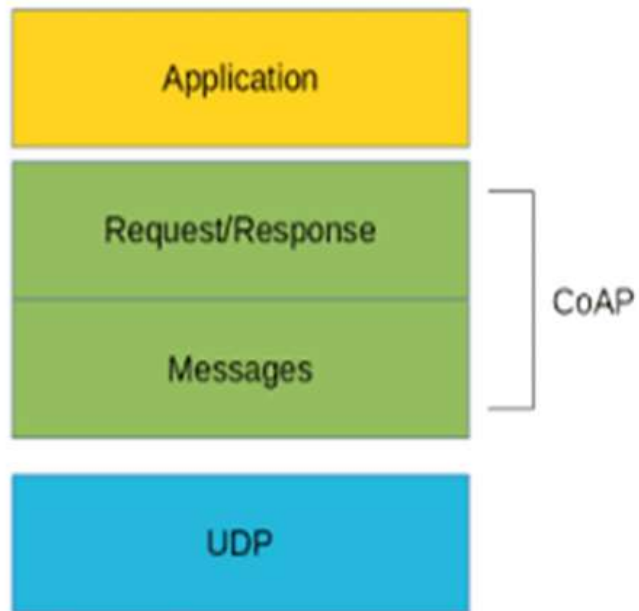
1. CoAP is more efficient in terms of bandwidth and energy consumption.
 2. It follows a client-server (request/response) mechanism. Thus, allow clients to perform CRUD operations on resources.
 3. It supports lightweight publish/subscribe model.
 4. It includes a resource observation feature that allows client to subscribe to a resource and receive notifications whenever the resource changes
-

CoAP Architecture



CoAP Layers

- ▶ Upper Layer is designed for communication method based on the Request-Response model.
- ▶ Lower Layer is designed to deal with UDP and asynchronous messages.



CoAP

- ▶ The devices in the network can act as a client or as a server.
- ▶ Each resource is identified by a URL, which allows clients to uniquely reference and interact with the resource.
- ▶ CoAP uses a request/response mechanism similar to HTTP.
- ▶ Requests are typically made using the methods GET, POST, PUT, and DELETE, which correspond to retrieving, creating, updating, and deleting resources, respectively.
- ▶ CoAP employs a lightweight message format based on the REST architectural style. Messages are exchanges using the UDP.
- ▶ CoAP message format consists of 4-byte header.

CoAP

- Four message types:
 - **Confirmable** – requires an ACK
 - **Non-confirmable** – no ACK needed
 - **Acknowledgement** – ACKs a Confirmable
 - **Reset** - indicates a Confirmable message has been received but context is missing for processing

CoAP message header

CoAP Message Header	Description
Ver	A 2-bit unsigned integer. It mentions the CoAP version number. Set to 1.
T	A 2-bit unsigned integer. Indicates message type, viz., confirmable (0), non-confirmable (1), ACK (2), or RESET (3).
TKL	A 4-bit unsigned integer, indicating the length of the token (0–8 bytes).
Code	An 8-bit unsigned integer, it is split into two parts, viz., 3-bit class (MSBs) and 5-bit detail (LSBs).
Message ID	A 16-bit unsigned integer, it is issued for matching responses and to detect message duplication.

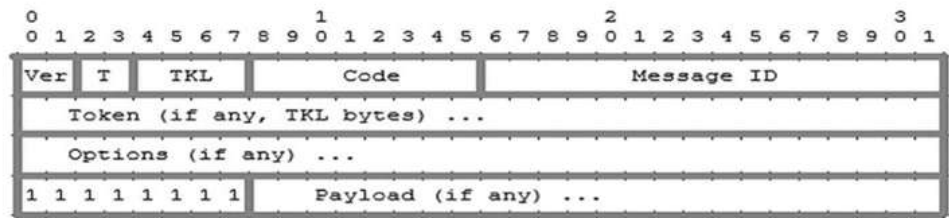
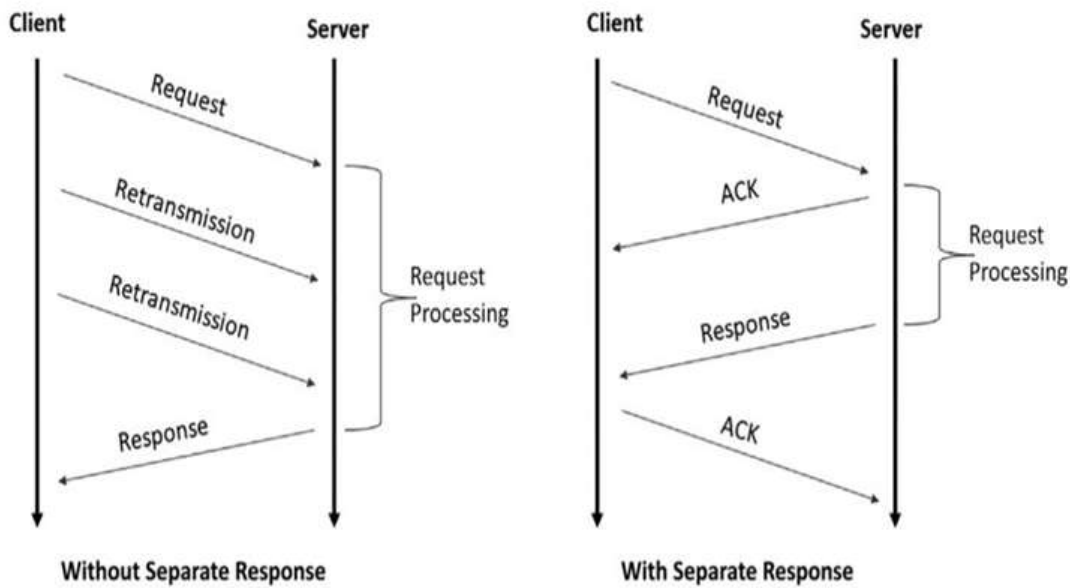


FIGURE 3.3 CoAP message header

CoAP exchange messages



♦ Two Types of CoAP Message Exchange

✓ 1. Without Separate Response (Left Side)

👉 Used when **server can respond quickly**

Steps:

1. Client sends **Request (CON message)**
2. If no reply → client **retransmits** (because UDP is unreliable)
3. Server processes request
4. Server sends **Response (with ACK)**

✓ Response itself acts as acknowledgment

👉 Simple meaning:

Request → (maybe retransmit) → Response

✓ 2. With Separate Response (Right Side)

👉 Used when **server needs more time**

Steps:

1. Client sends **Request (CON)**
2. Server immediately sends **ACK** (just to confirm receipt)
3. Server processes request (takes time)
4. Server sends **Response later (CON)**
5. Client sends **ACK for response**

👉 Simple meaning:

Request → ACK → (processing) → Response → ACK

◆ One-Line Concept

👉 Without separate = immediate reply

👉 With separate = delayed reply with confirmation

MQTT (Message Queuing Telemetry Transport)

◆ What is MQTT (Simple Idea)

👉 MQTT = lightweight messaging protocol for IoT using publish/subscribe model

- Works on **low bandwidth & low power devices**
 - Uses a **broker (middleman)**
 - Best for **IoT, sensors, ESP32, mobile apps**
-

◆ Basic Concept (Very Important)

No direct communication between devices

Instead:

- **Publisher → sends data**
- **Broker → manages data**
- **Subscriber → receives data**

👉 Think like:

📡 Sender → 📦 Broker → 📱 Receiver

◇ Main Components

1. Publisher

- Sends data
- Example: temperature sensor

2. Subscriber

- Receives data
- Example: mobile app

3. Broker (most important)

- Middleman
 - Receives & forwards messages
-

◆ Step-by-Step MQTT Working

✓ Step 1: Choose an MQTT Broker:

- Broker = central server
- Examples: Mosquitto, HiveMQ

Role:

- Receives messages
 - Sends to correct users
 - Manages communication
-

✓ Step 2: Choose an MQTT Client Library:

MQTT client libraries are available in several programming languages, including Python, Java, C, and C++. You can choose a client library that is compatible with your IoT device and programming language.

✅ Step 3: Connect Your Device to the MQTT Broker:

Connect Device to Broker

- Device connects using:
 - **IP address**
 - **Port (usually 1883)**
 - **Username/Password**

You will need to provide the broker's address, port number, and credentials, such as username and password.

Example:

ESP32 connects to MQTT broker

✅ Step 4: Publish and Subscribe to MQTT Topics

Create Topics

- Topic = channel name

👉 Example:

home/room1/temp

home/led

✓ Used to organize data

- You can publish and subscribe to MQTT topics using the client library
-

✅ Publish Data

- Publisher sends data to a topic

👉 Example:

Sensor publishes:

Topic: home/temp

Message: 25°C

✓ **Subscribe to Topic**

- Subscriber listens to topic

👉 Example:

Mobile app subscribes:

Topic: home/temp

✓ Whenever data comes → it receives automatically

✓ **Broker Sends Data**

- Broker receives message
- Sends to all subscribers of that topic

👉 Flow:

Publisher → Broker → Subscriber

✓ **Step 5: Test Your MQTT Setup:**

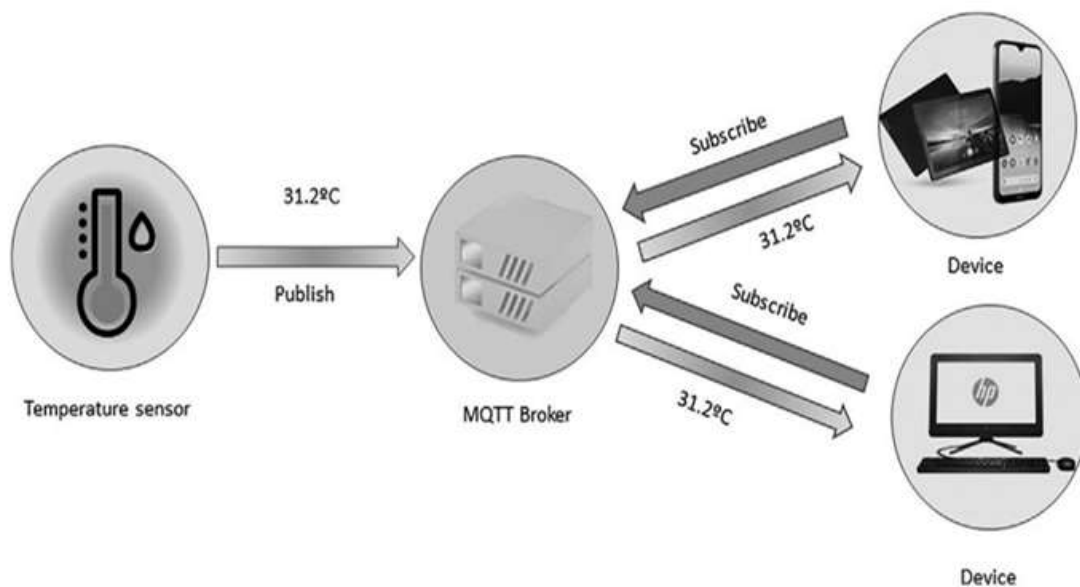
Once you have connected your device to the MQTT broker and published and subscribed to topics, you can test your setup by sending and receiving messages.

- Send messages
 - Check if subscriber receives
-

MQTT (Message Queuing Telemetry Transport)

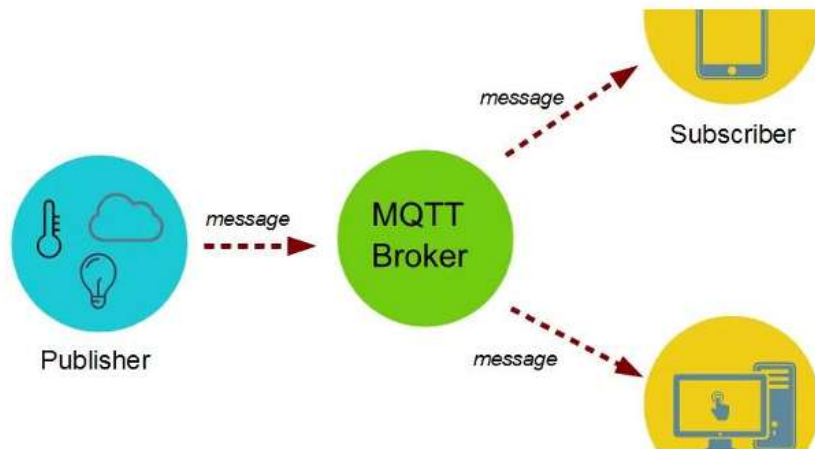
- ▶ It is a lightweight publish/subscribe messaging protocol.
- ▶ It is designed to provide reliable, efficient, and secure communication in resource constrained environments.
- ▶ It is a M2M connectivity protocol and is useful for connections with remote locations where network bandwidth is premium.
- ▶ It communicate with applications through the MQTT message broker.
- ▶ It runs over TCP protocol.
- ▶ MQTT can be used to connect large numbers of devices, making it an ideal choice for IoT applications that require scalability.

MQTT Architecture

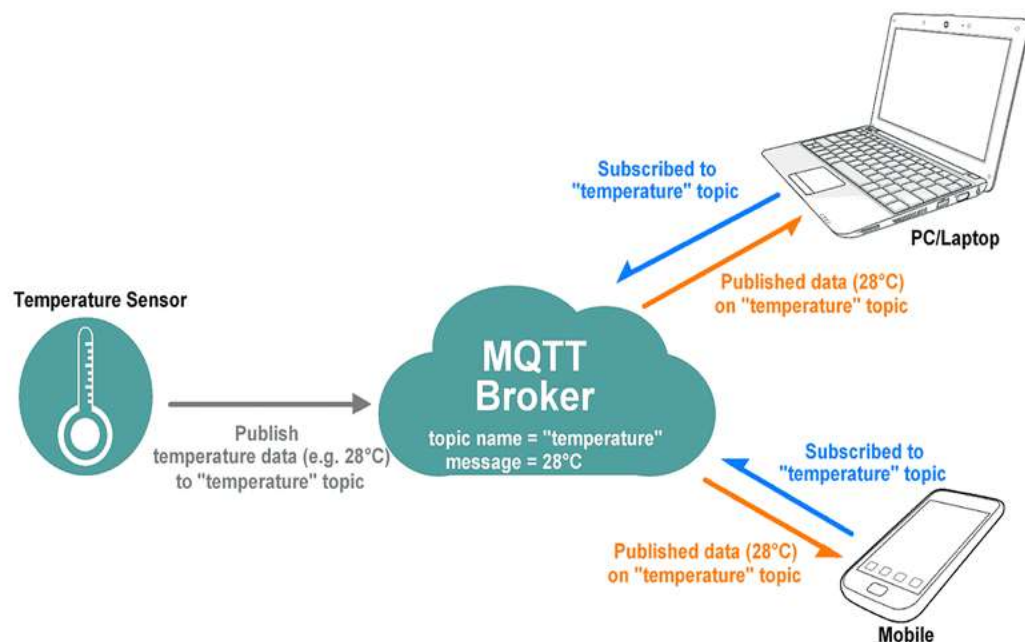


MQTT Components

1. Broker
2. Topic
3. Message
4. Client
 - Publisher
 - Subscriber



Working of MQTT



◇ Role of MQTT Broker (Very Important)

👉 Broker = **heart of MQTT system**

Functions:

1. Accepts connections (publisher & subscriber)
 2. Manages **topics & subscriptions**
 3. Routes messages correctly
 4. Controls **QoS (reliability levels)**
 5. Handles **security (authentication)**
 6. Logs and monitors data
-

◆ MQTT Applications

- 🏭 Industrial automation
- 🏠 Smart homes
- 🏥 Healthcare
- 🚗 Transportation
- ⚡ Energy systems
- 🌾 Agriculture

MQTT vs CoAP (Easy Comparison)

Feature	MQTT	CoAP
Model	Publish–Subscribe	Request–Response (+ observe)
Transport	TCP	UDP
Reliability	QoS (0,1,2 levels)	ACK + retransmission
Architecture	Needs Broker	Direct Client–Server
Speed	Slower (due to TCP)	Faster (UDP)
Power Usage	Low	Very Low
Best Use	Data streaming, monitoring Quick control, REST APIs	

MQTT vs CoAP

	MQTT	CoAP
Application Layer	Single Layered completely	Single Layered with 2 conceptual sub layers (Messages Layer and Request Response Layer)
Transport Layer	Runs on TCP	Runs on UDP
Reliability Mechanism	3 Quality of Services levels	Confirmable messages, Non-confirmable messages, Acknowledgments and retransmissions
Supported Architectures	Publish-Subscribe	Request-Response, Resource observe/Publish-Subscribe

◇ Simple Difference (One Line)

👉 MQTT = Broker-based communication (indirect)

👉 CoAP = Direct communication (like HTTP)

◇ When to Use What?

✓ Use **MQTT** when:

- Many devices
- Continuous data (sensor data)
- Need reliability

✓ Use **CoAP** when:

- Fast response needed
 - Low power devices
 - Simple request/response
-

◇ Final Concept (Exam Line)

👉 MQTT is best for scalable IoT messaging via broker, while CoAP is best for lightweight, fast, direct device communication

MQTT is used in a wide variety of industries and applications, including the following

1. Industrial Automation:

MQTT is used in industrial automation applications to enable communication between sensors, actuators, and control 155 Wireless Protocols and Technologies for the Internet of Things systems. It can be used to monitor and control machines and processes, and to collect data for analysis and optimization.

2. Smart Home and Building Automation:

MQTT is used in smart home and building automation applications to enable communication between devices such as smart thermostats, lighting systems, and security systems. It can be used to control devices and monitor their status, and to provide real-time data to users.

3. Healthcare:

MQTT is used in healthcare applications to enable communication between medical devices, patient monitoring systems, and healthcare applications. It can be used to collect data from medical devices and transmit it securely to healthcare applications for analysis and diagnosis.

4. Transportation:

MQTT is used in transportation applications to enable communication between vehicles, traffic management systems, and transportation applications. It can be used to collect data on traffic conditions, vehicle performance, and driver behavior, and to optimize traffic flow and reduce congestion.

5. Energy and Utilities:

MQTT is used in energy and utilities applications to enable communication between smart meters, energy management systems, and utility applications. It can be used to collect data on energy consumption, optimize energy usage, and improve the reliability and efficiency of energy systems.

6. Agriculture:

MQTT is used in agriculture applications to enable communication between sensors, actuators, and agricultural management systems. It can be used to monitor soil moisture, temperature, and other environmental factors, and to optimize irrigation, fertilization, and other agricultural processes.